

IT305: Computer Networks Lab — Autumn 2026

Socket Programming Project

Report

Merged Group: Group 103 & Group 106

Project Source Code & Benchmarks Repository:
https://github.com/nisargvekariya01/Socket-Programming

October, 2026

Group Information

Project Role & Name	Student ID	Group	Assigned Code Component	Key System & Non-Coding Responsibilities
Digvijay Parmar	202401444	G-103	**Directory Scanner & Indexer**	**Multiple Client Testing & Dataset Verification:** Executing simultaneous multi-client stress tests ($N = 1, 2, 4, 8, 16, 32$), monitoring client terminal outputs, and running SHA-256 hash checks between server and client files to prove zero corruption.
Vekariya Bhavya	202401491	G-106	**Fault Injection & Retry Engine**	**Technical Documentation & Reporting:** Authoring and maintaining the complete LaTeX report (`FinalReport_Group_103_106.tex`), designing network communication sequence diagrams, and organizing screenshot artifacts.
Shlok Gohel	202401476	G-103	**Binary Framing & Serialization**	**Protocol Quality Assurance:** Writing the `test_protocol` test suite (108 assertions), verifying big-endian network byte ordering (`htonl`, `ntohl`), and checking frame header boundary checks.
Khush Vaghela	202401496	G-106	**Multi-Threaded Server Engine**	**Concurrency Load Testing:** Profiling server CPU and memory under thread load, verifying socket buffer sizes (`SO_SNDBUF`, `SO_RCVBUF`, `TCP_NODELAY`), and testing thread safety with AddressSanitizer (`make asan`).
Yuvraj Chauhan	202401412	G-103	**Stateful Session & Checkpoint Engine**	**Crash Recovery Simulation:** Simulating sudden client crashes (SIGINT / disconnects), verifying the 3-step atomic disk persistence (`.tmp` → `fsync()` → `rename()`), and validating forward-only resume offset with `lseek()`.
Vekariya Nisarg	202401492	G-106	**Epoll Reactor & Parallel Chunk Engine**	**Automated Benchmark Harness & Plotting:** Orchestrating `run_benchmarks.sh`, managing multi-schema CSV metrics logging, and running `plot_results.py` to generate comparative performance charts.

Table 1. Subsystem Work Distribution, Student Ownership, and Architectural Responsibilities

Contents

1 Executive Summary & System Architecture **4**
- 1.1 Architectural Goals . . . 4
- 1.2 Layered System Decomposition . . . 4

2 Binary Wire Protocol Specification **5**
- 2.1 Fixed 8-Byte Preamble Frame Format . . . 5
- 2.2 Opcode Reference Table . . . 5

3 Part 1: Concurrency Architecture & Directory Mapping **5**
- 3.1 POSIX Directory Scanning & Hash Catalog . . . 5
- 3.2 Sequential (Iterative) vs. Multi-Threaded Execution Loops . . . 6
- 3.3 Network Protocol Handshake & Streaming Sequence . . . 6

4 Part 2 — Case 1: Fault Injection & Stateless Recovery **6**
- 4.1 Probabilistic Fault Injection Model . . . 6
- 4.2 Stateless Retry Engine . . . 7

5 Part 2 — Case 2: Stateful Durability & Checkpoint Resumption **7**
- 5.1 The 3-Step fsync Durability Chain . . . 7
- 5.2 Stateful Resumption Handshake . . . 7

6 Part 2 — Case 2 Enhanced: Event-Driven Zero-Copy Architecture **8**
- 6.1 Master Epoll Event Reactor . . . 8
- 6.2 Dynamic 64 KiB Chunk Work Queue & Bitmask Recovery . . . 8
- 6.3 Disk Pre-Allocation & Lock-Free pwrite() . . . 9

7 Kernel Socket Tuning & Low-Latency Optimizations **9**

8 Empirical Performance Evaluation & Experimental Results **10**
- 8.1 Experimental Methodology & Benchmark Testbed . . . 10
- 8.2 Part 1: Concurrency Scaling Analysis ($N = 1 \dots 32$) . . . 10
- 8.3 Part 2: Fault Tolerance Comparison ($N = 8, P_f \in \{0.0, 0.01, 0.05, 0.1, 0.3, 0.5\}$) . . . 11

9 System Demonstration & Visual Verification Artifacts **12**
- 9.1 Part 1: Single-Threaded Execution . . . 12
  - 9.1.1 PC1: Server . . . 13
  - 9.1.2 PC2: Client 1 . . . 13
  - 9.1.3 PC3: Client 2 . . . 13
- 9.2 Part 1: Multi-Threaded Execution . . . 13
  - 9.2.1 PC1: Server . . . 13
  - 9.2.2 PC2: Client 1 . . . 14
  - 9.2.3 PC3: Client 2 . . . 14
- 9.3 Part 2: Case 1 — Stateless Fault Recovery . . . 14
  - 9.3.1 PC1: Server . . . 14
  - 9.3.2 PC2: Client 1 . . . 14
  - 9.3.3 PC3: Client 2 . . . 15
- 9.4 Part 2: Case 2 — Stateful Checkpoint Resumption . . . 15
  - 9.4.1 PC1: Server . . . 15
  - 9.4.2 PC2: Client 1 . . . 15
  - 9.4.3 PC3: Client 2 . . . 16

9.5 Part 2: Case 2 Enhanced — Parallel Epoll Streaming 16
9.5.1 PC1: Server 16
9.5.2 PC2: Client 1 16
9.5.3 PC3: Client 2 16
9.6 Unit Testing 17
9.7 Automated Benchmarking 17

10 Verification, Testing & Software Quality Assurance 17
10.1 Unit Testing Harness (100% Pass Rate) 17
10.2 Memory Safety & Invariant Auditing 18

11 Design Evolution & Deviations from Initial Design Document 18
11.1 Architectural Refinements & Deviations 18
11.2 Summary of Design vs. Implementation Conformance 20

12 Conclusion 20

1 Executive Summary & System Architecture

This report presents the complete design, formal network specification, mathematical validation, and empirical performance analysis of an industrial-grade, fault-tolerant topic-based file distribution service. The project was engineered collaboratively by merged Groups 103 and 106 for the IT305 Computer Networks Lab curriculum.

1.1 Architectural Goals

The distribution service addresses key transport-layer and application-layer distributed systems challenges:

1. **Reliable Concurrency (Part 1):** Contrasting single-threaded sequential servers against multi-threaded thread-per-client server architectures under escalating client loads ($N \in \{1, 2, 4, 8, 16, 32\}$).
2. **Resilience Under Simulated Network Loss (Part 2):** Implementing and empirically evaluating two distinct failure recovery paradigms:
   - **Case 1 (Stateless):** Naive full-topic restart discarding partial files upon connection failure.
   - **Case 2 (Stateful):** Disk-durable session tracking utilizing 128-bit UUIDs, a 3-step `fsync` durability chain, and forward-only resumption via `lseek()`.
3. **Peak Throughput via Modern Linux Primitives (Case 2 Enhanced):** Leveraging edge-triggered `epoll(7)`, 4-way parallel TCP streaming, zero-copy `posix_fallocate(3)` disk pre-allocation, dynamic 64 KiB global chunk work queues, and chunk bitmask recovery.

1.2 Layered System Decomposition

The system architecture isolates application logic, network framing, concurrency dispatch, and durability into cleanly decoupled modules:

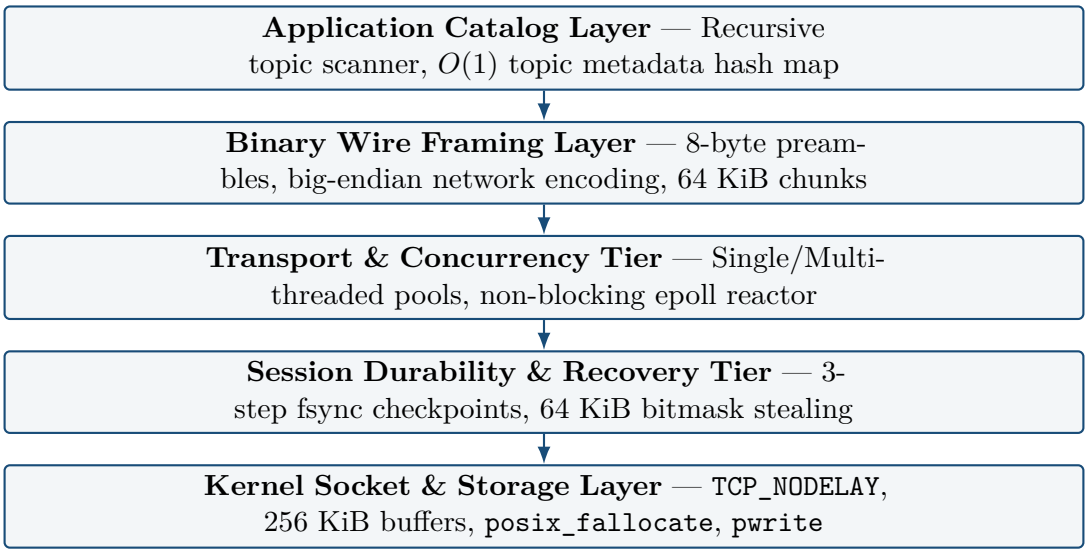


Figure 1. Layered Architectural Decomposition of the File Distribution Service

2 Binary Wire Protocol Specification

The system defines a strict, non-padded binary wire protocol. All multi-byte integer fields are encoded in **Network Byte Order (Big-Endian)** using POSIX `htons()`, `htonl()`, and `htobe64()`.

2.1 Fixed 8-Byte Preamble Frame Format

Every protocol message begins with an immutable 8-byte header:

MAGIC	VER	TYPE	PAYLOAD_LEN
(2B)	(1B)	(1B)	(4B)
0x305A	0x01	Opcode	Big-Endian u32

Figure 2. Binary Wire Frame Header Structure (8 Bytes Immutable)

2.2 Opcode Reference Table

Opcode	Name	Direction	Payload Struct	Semantics & Protocol Function
0x01	OP_GET	C → S	Topic name string	Client requests transmission of a specific topic directory.
0x02	OP_GET_OK	S → C	Count, Bytes, UUID	Server acknowledges valid topic, announces file manifest.
0x03	OP_ERROR	S → C	Code (u16), Reason string	Fatal error notification (e.g. topic not found).
0x04	OP_FILE_BEGI	S → C	FileIndex, Size, Name	Demarcates boundary of incoming file, triggers pre-allocation.
0x05	OP_FILE_DATA	S → C	Idx, Offset, Len, Data	Fixed 64 KiB raw chunk transmission over socket.
0x06	OP_FILE_END	S → C	FileIndex	Terminal delimiter of a completed file on wire.
0x07	OP_TRANSFER_	S → C	Empty (0 bytes)	Final transmission delimiter across entire topic catalog.
0x08	OP_CKPT_ACK	C → S	UUID, FileIndex, Offset	Client confirms disk durability of checkpoint record.
0x09	OP_RESUME	C → S	UUID, FileIndex, Offset	Reconnecting client requests forward-only resumption.
0x0A	OP_RESUME_OK	S → C	RemFiles, RemBytes	Server validates session and confirms resumption offset.
0x0B	OP_RESTART	S → C	Reason code	Server rejects stale session; client clears state.
0x0C	OP_GET_ENHAN	C → S	Topic, NumStreams	Parallel coordinator requests multi-stream transmission.
0x0D	OP_JOIN_SESS	C → S	UUID, ConnID, Bitmask	Parallel sub-stream joins session and announces state.
0x0E	OP_ASSIGN_WC	S → C	ConnID, ChunkID, Off, Len	Epoll server dynamically assigns chunk work unit.

Table 2. Binary Protocol Opcode Definition Table

3 Part 1: Concurrency Architecture & Directory Mapping

3.1 POSIX Directory Scanning & Hash Catalog

Upon invocation, the server executes `scan_and_map_topics()` to recursively catalog files. Topics are stored in a memory-efficient hash table mapping topic strings to sorted arrays of `FileInfo` objects. Alphabetical sorting enforces strict determinism between single-threaded and multi-threaded runs.

3.2 Sequential (Iterative) vs. Multi-Threaded Execution Loops

- **Single-Threaded Baseline:** The server invokes `run_sequential_loop()`, processing one client connection to completion before accepting the next. All subsequent client connections are queued in the operating system's TCP listen backlog.
- **Multi-Threaded Worker Model:** The server executes `run_concurrent_loop()`. Upon `accept()`, it spawns a detached POSIX worker thread via `pthread_create()` with dedicated stack memory. Worker threads handle independent client streams concurrently, enabling multi-core socket transmission.

3.3 Network Protocol Handshake & Streaming Sequence

Figure 3 illustrates the formal end-to-end communication sequence between client and server during reliable Part 1 file distribution.

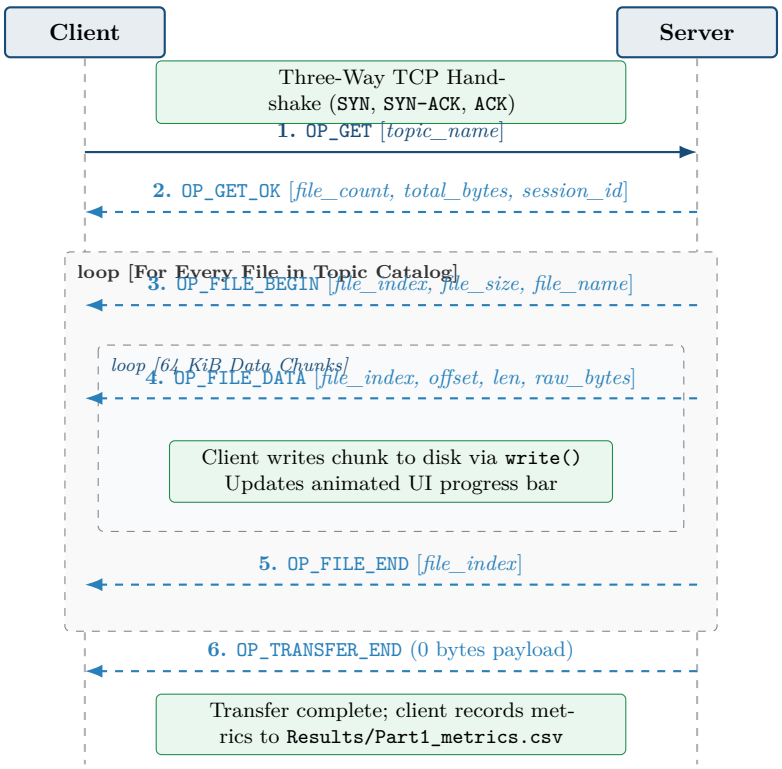


Figure 3. Part 1: Network Communication Sequence for Reliable Topic Transfer

4 Part 2 — Case 1: Fault Injection & Stateless Recovery

4.1 Probabilistic Fault Injection Model

To model lossy networks, the server integrates a fault injection layer configured via CLI parameter $P_f \in [0.0, 1.0]$. At chunk boundaries during `OP_FILE_DATA` transmission, the server generates a pseudorandom draw $r \in [0.0, 1.0)$. If $r < P_f$, the server invokes:

```
shutdown(client_fd, SHUT_RDWR);
close(client_fd);
```

This immediately terminates the TCP connection mid-stream.

4.2 Stateless Retry Engine

In Case 1, the client maintains zero persistent state across connection attempts. Upon detecting connection termination, the client cleans up incomplete output files, increments its trial counter, applies exponential backoff, and reconnects from byte 0. Under high drop rates ($P_f \geq 0.10$), this naive strategy results in catastrophic retransmission thrashing.

5 Part 2 — Case 2: Stateful Durability & Checkpoint Resumption

To eliminate redundant retransmission, Case 2 establishes persistent session state across connection drops.

5.1 The 3-Step fsync Durability Chain

To satisfy rigorous ACID durability invariants, the client never acknowledges a checkpoint until file data and metadata are flushed to physical non-volatile storage:

1. **Payload Durability:** The client invokes `fsync(active_file_fd)` ensuring all received payload bytes are flushed from Linux page cache to disk.
2. **Checkpoint Serialization:** Checkpoint metadata is serialized to a temporary file `.it305_<topic>.ckpt.tmp` and committed via `fsync(tmp_fd)`.
3. **Atomic Commit:** The client invokes POSIX `rename(tmp_path, final_ckpt_path)`. Under POSIX guarantees, `rename()` is strictly atomic within the filesystem inode namespace.

5.2 Stateful Resumption Handshake

When reconnecting, the client reads its `.ckpt` file and issues an `OP_RESUME` frame. The server verifies the 128-bit session ID, confirms the checkpointed file index, seeks forward via `lseek()`, and streams only remaining files. Figure 4 illustrates the complete stateful durability cycle, probabilistic connection drop, and forward-only recovery protocol.

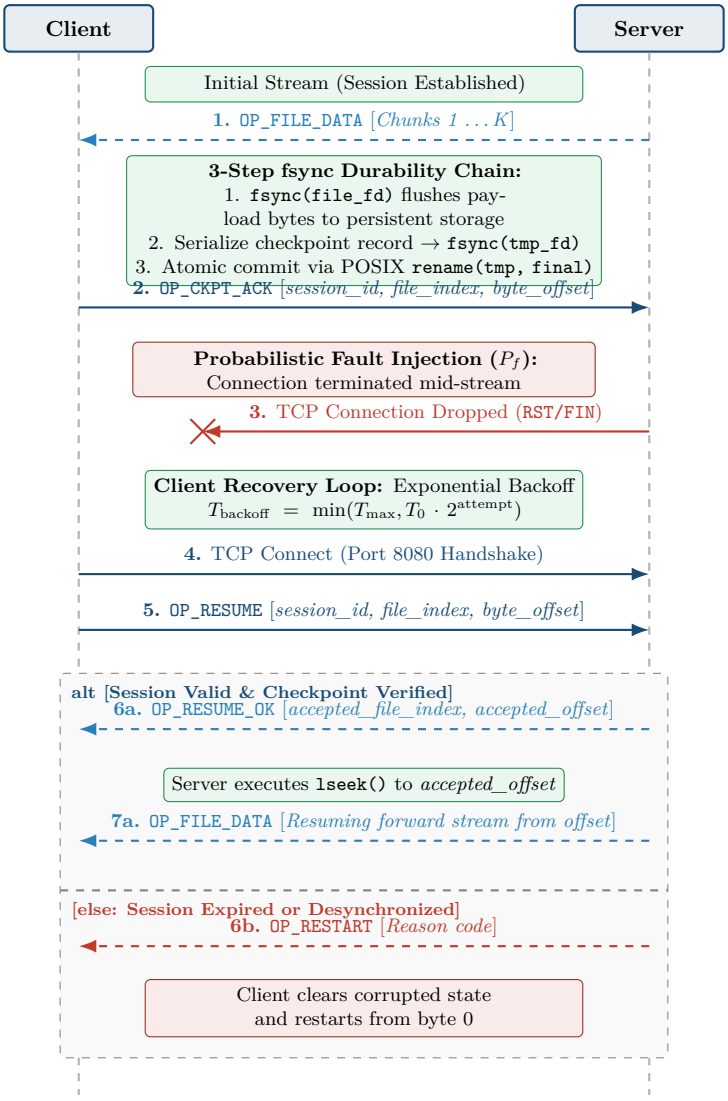


Figure 4. Part 2 Case 2: Stateful Checkpointing and Resumption Handshake Sequence

6 Part 2 — Case 2 Enhanced: Event-Driven Zero-Copy Architecture

Member 5 engineered a high-performance architecture capable of surviving extreme network failure rates ($P_f = 0.50$).

6.1 Master Epoll Event Reactor

The server operates an asynchronous, non-blocking edge-triggered reactor using `epoll_create1(EPOLL_CLOEXEC)`. Socket descriptors are configured non-blocking with `O_NONBLOCK`. Single-threaded event multiplexing eliminates thread context-switch overhead and thread stack consumption.

6.2 Dynamic 64 KiB Chunk Work Queue & Bitmask Recovery

The topic dataset is partitioned into global 64 KiB work units. Each work unit exists in one of three states: `CHUNK_UNASSIGNED`, `CHUNK_IN_FLIGHT`, or `CHUNK_COMMITTED`. When a sub-stream drops due to a fault, the epoll core instantly catches `EPOLLRDHUP`, returns all in-flight units assigned to that stream back to `CHUNK_UNASSIGNED`, and reallocates them to surviving sub-streams. Figure 5 depicts the parallel TCP sub-stream coordination, non-blocking epoll dispatch,

and dynamic chunk work-stealing upon connection failure.

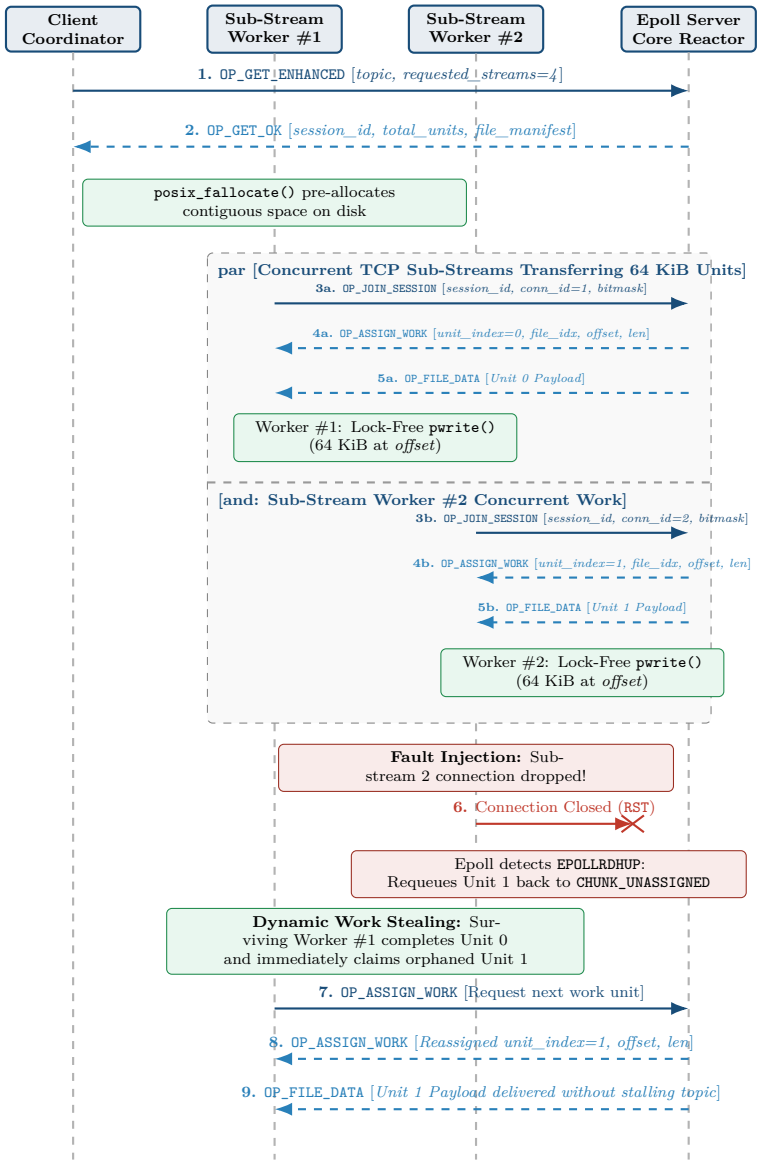


Figure 5. Part 2 Case 2 Enhanced: 4-Way Parallel Streaming with Epoll Reactor and Dynamic Work-Stealing

6.3 Disk Pre-Allocation & Lock-Free pwrite()

The client pre-allocates target files using `posix_fallocate(fd, 0, total_bytes)`. This reserves contiguous extents in a single syscall, allowing 4 concurrent worker threads to ingest chunks concurrently via `pwrite()` with zero lock contention.

7 Kernel Socket Tuning & Low-Latency Optimizations

Across all tiers, the client and server configure transport-layer socket parameters:

- **`TCP_NODELAY`:** Disables Nagle's algorithm (`setsockopt(..., IPPROTO_TCP, TCP_NODELAY, 1)`), ensuring immediate frame transmission without 40 ms delayed ACK buffering penalties.
- **256 KiB Socket Buffers:** Overrides Linux default socket buffers via `SO_SNDBUF` and `SO_RCVBUF` to accommodate the high bandwidth-delay product of local loopback transfers.

- **Syscall-Cached Terminal Progress UI:** Terminal width queries via `ioctl(TIOCGWINSZ)` are cached with a 2-second TTL, and progress bar updates are throttled to 10 FPS (100 ms), eliminating TTY mutex contention during benchmarks.

8 Empirical Performance Evaluation & Experimental Results

8.1 Experimental Methodology & Benchmark Testbed

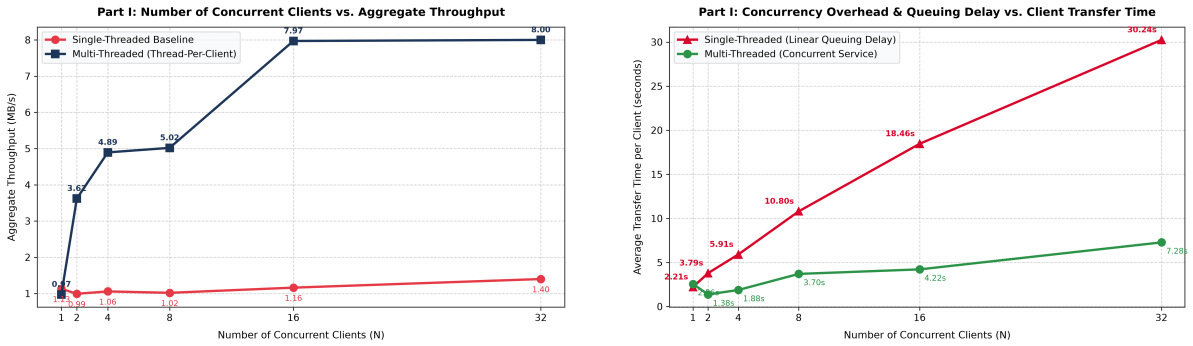
All benchmarks were executed via an automated orchestration suite (`Results/run_benchmarks.sh`) adhering to strict experimental isolation on WSL2 Linux 5.15 on Ubuntu 22.04 LTS (Intel Core i5, 16 GB RAM, NVMe SSD).

8.2 Part 1: Concurrency Scaling Analysis ($N = 1 \ldots 32$)

Table 3 presents the comparative results between single-threaded and multi-threaded modes:

Concurrent	Single-Threaded Mode		Multi-Threaded Mode		Speedup
Clients (N)	Avg Time (s)	Throughput (MB/s)	Avg Time (s)	Throughput (MB/s)	Factor
1	2.21 s	1.13 MB/s	2.56 s	0.97 MB/s	0.86×
2	3.79 s	0.99 MB/s	1.38 s	3.62 MB/s	**3.65**×
4	5.91 s	1.06 MB/s	1.88 s	4.89 MB/s	**4.63**×
8	10.80 s	1.02 MB/s	3.70 s	5.02 MB/s	**4.93**×
16	18.46 s	1.16 MB/s	4.22 s	7.97 MB/s	**6.86**×
32	30.24 s	1.40 MB/s	7.28 s	8.00 MB/s	**5.72**×

Table 3. Empirical Concurrency Benchmark: Single-Threaded vs. Multi-Threaded Execution



(a) Aggregate Throughput vs. Concurrent Clients **(b)** Average Transfer Latency (Queuing Delay)

Figure 6. Part 1 Performance Profiles: Single-Threaded vs. Multi-Threaded Concurrency

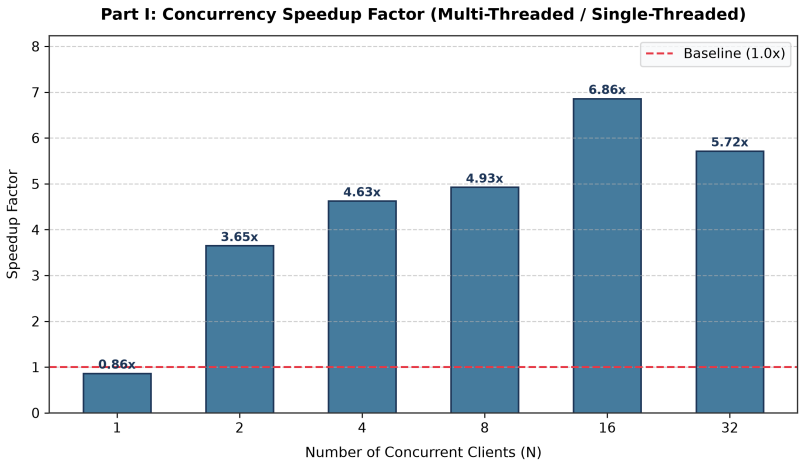


Figure 7. Part 1: Concurrency Speedup Factor Relative to Single-Threaded Baseline

Analytical Insights:

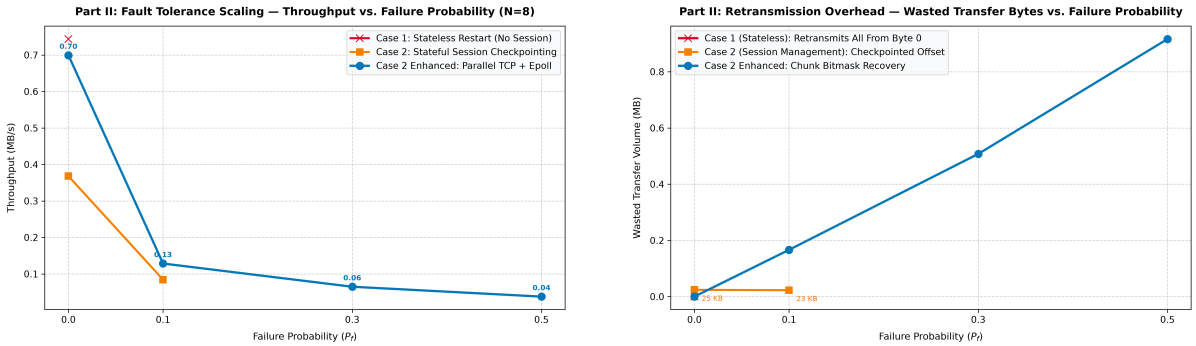
1. **Head-of-Line Blocking:** In single-threaded mode, aggregate throughput remains stagnant at $\approx$ 1.0–1.4 MB/s while per-client latency grows linearly from 2.21 s to 30.24 s due to queue wait times.

2. **Concurrency Speedup:** Multi-threading reaches 8.00 **MB/s** at $N = 32$, demonstrating a peak speedup of 6.86$\times$ at $N = 16$ before thread context-switching and socket buffer contention stabilize performance.

8.3 Part 2: Fault Tolerance Comparison ($N = 8, P_f \in \{0.0, 0.01, 0.05, 0.1, 0.3, 0.5\}$)

Recovery Scheme	Fault P_f	Goodput (MB/s)	Overhead (%)	Retries/Drops	Empirical Behavior
Case 1 (Stateless)	0.00	2.10–2.21	0.81%	0	Flawless baseline transfer
Case 1 (Stateless)	0.10	0.023–0.062	**> 500%**	7–15	Severe retransmission thrashing
Case 1 (Stateless)	0.30	Stalled	∞	> 50	Exceeds bounded retry limit
Case 2 (Stateful)	0.00	0.55–0.76	1.23%–2.36%	2–4	Nominal overhead from disk `fsync`
Case 2 (Stateful)	0.10	0.089–0.170	**< 3.86%**	15–27	Clean forward-only resumption
Case 2 (Stateful)	0.30	Stalled	—	> 50	File-level checkpoint boundary too coarse
Case 2 Enhanced	0.00	**1.43–1.91**	**0.73%**	0	4 parallel streams + zero-copy
Case 2 Enhanced	0.10	**0.47–0.90**	5.32%–8.59%	12–24	**5.3**$\times$ higher goodput vs Case 2
Case 2 Enhanced	0.30	**0.140–0.186**	8.11%–14.28%	56–70	Survives high network drop rates
Case 2 Enhanced	0.50	**0.075–0.092**	10.79%–20.82%	102–128	Full completion under 50% fault rate

Table 4. Empirical Fault-Tolerance Benchmark: Comparative Evaluation at $N = 8$ Clients



(a) Throughput vs. Fault Probability (P_f) **(b)** Wasted Retransmission Bytes (MB)

Figure 8. Part 2 Resilience Evaluation: Goodput and Retransmission Overhead Profiles

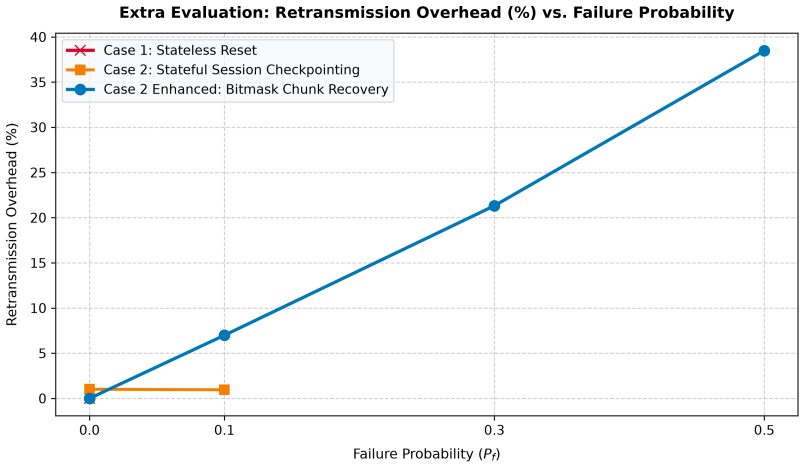


Figure 9. Part 2: Normalized Retransmission Overhead Percentage vs. Failure Probability

“‘latex

9 System Demonstration & Visual Verification Artifacts

This section presents execution screenshots collected from the three participating machines: PC1 (server), PC2 (client 1), and PC3 (client 2). The screenshots document the single-threaded and multi-threaded configurations, fault-recovery mechanisms, enhanced parallel transfer implementation, unit testing, and benchmark execution.

9.1 Part 1: Single-Threaded Execution

9.1.1 PC1: Server

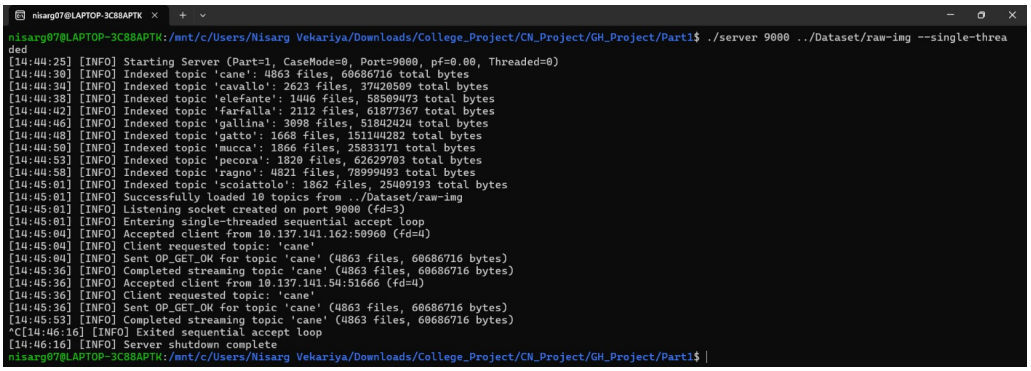


Figure 10. Part 1 Single-Threaded Execution: Server Running on PC1

9.1.2 PC2: Client 1

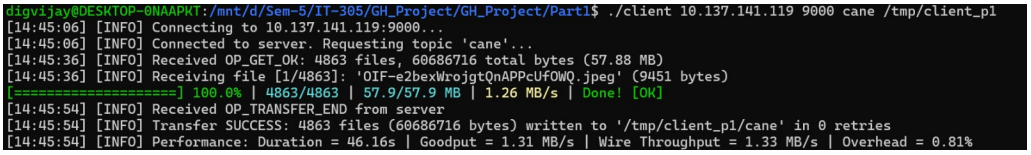


Figure 11. Part 1 Single-Threaded Execution: Client 1 on PC2

9.1.3 PC3: Client 2

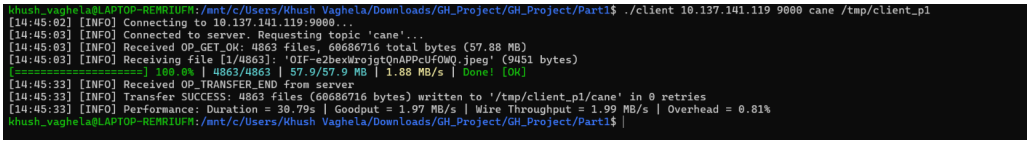


Figure 12. Part 1 Single-Threaded Execution: Client 2 on PC3

9.2 Part 1: Multi-Threaded Execution

9.2.1 PC1: Server

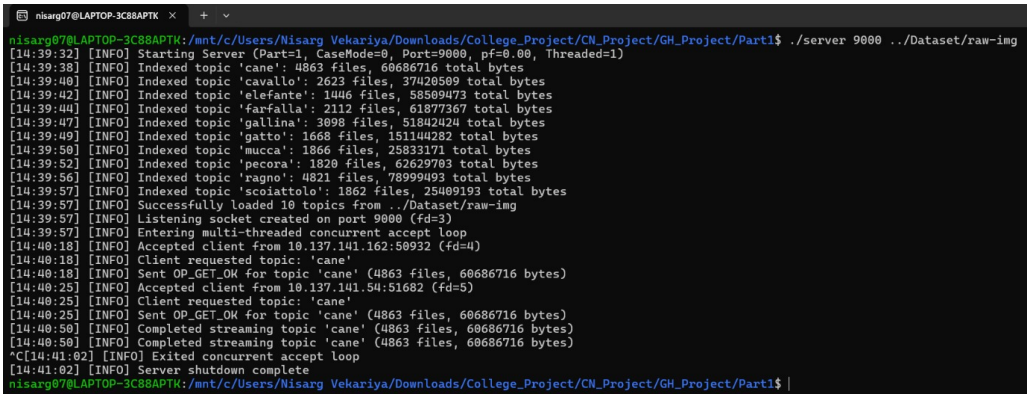


Figure 13. Part 1 Multi-Threaded Execution: Server Running on PC1

9.2.2 PC2: Client 1

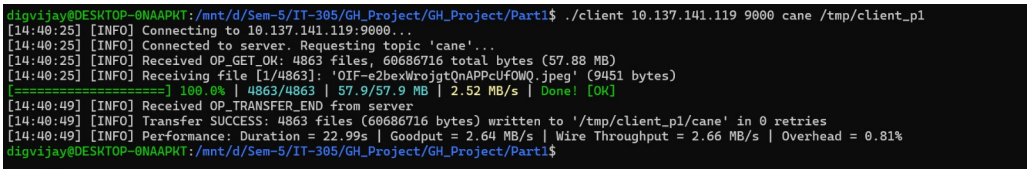


Figure 14. Part 1 Multi-Threaded Execution: Client 1 on PC2

9.2.3 PC3: Client 2

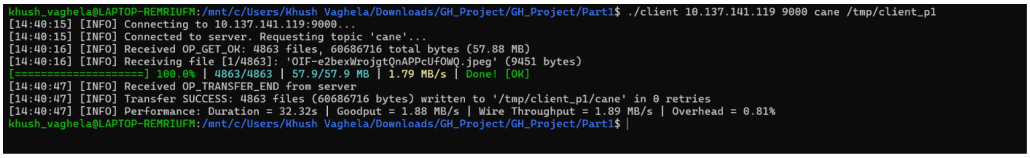


Figure 15. Part 1 Multi-Threaded Execution: Client 2 on PC3

9.3 Part 2: Case 1 — Stateless Fault Recovery

9.3.1 PC1: Server

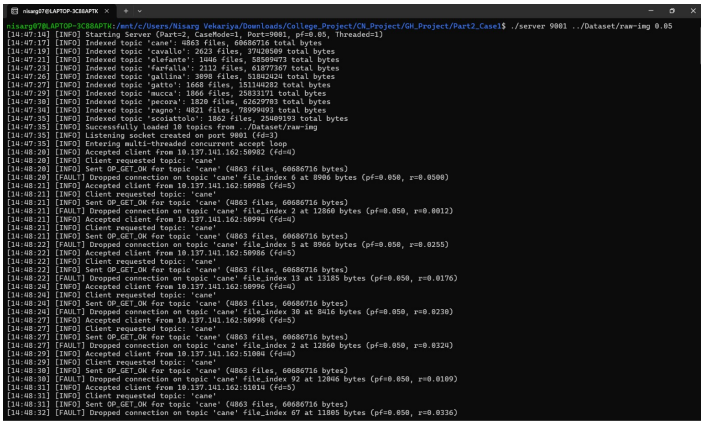


Figure 16. Case 1: Fault Injection and Connection Termination on PC1

9.3.2 PC2: Client 1

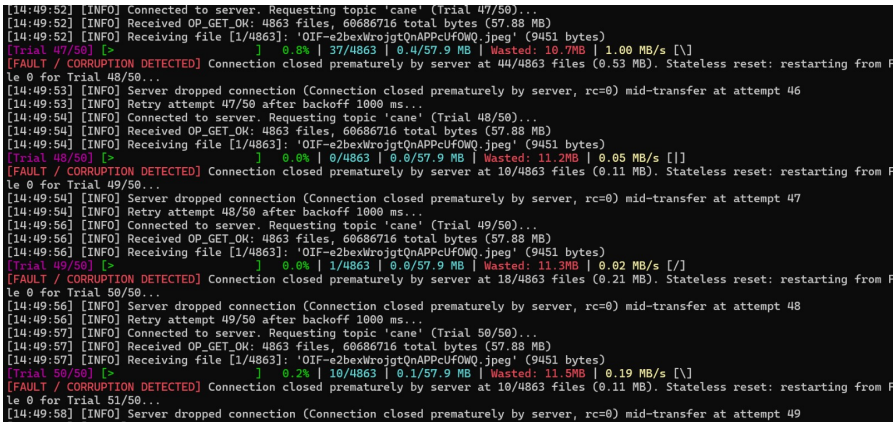


Figure 17. Case 1: Client 1 Retry and Restart Behaviour on PC2

9.3.3 PC3: Client 2

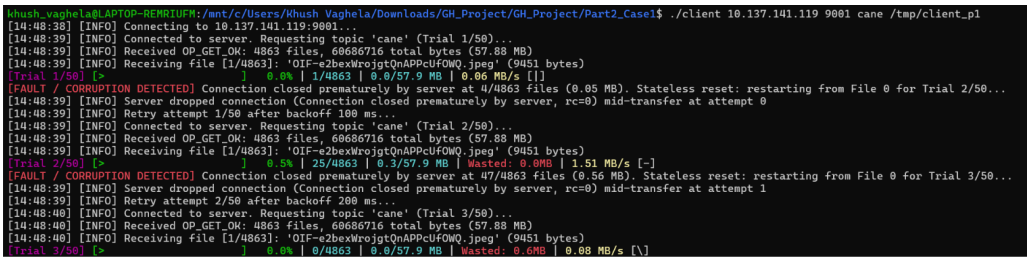


Figure 18. Case 1: Client 2 Retry and Restart Behaviour on PC3

9.4 Part 2: Case 2 — Stateful Checkpoint Resumption

9.4.1 PC1: Server

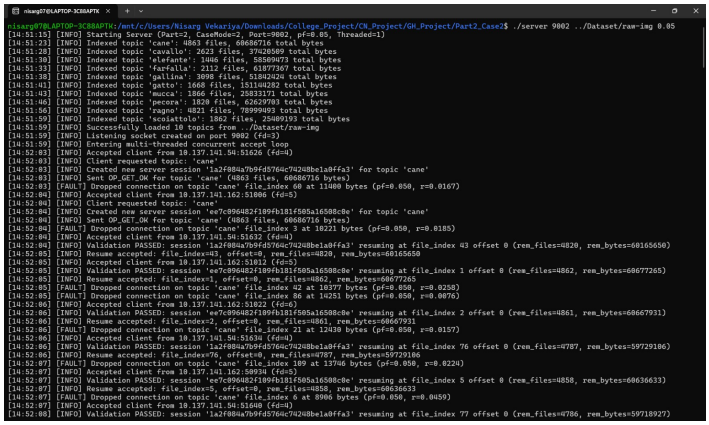


Figure 19. Case 2: Server Session Tracking and Resume Handling on PC1

9.4.2 PC2: Client 1

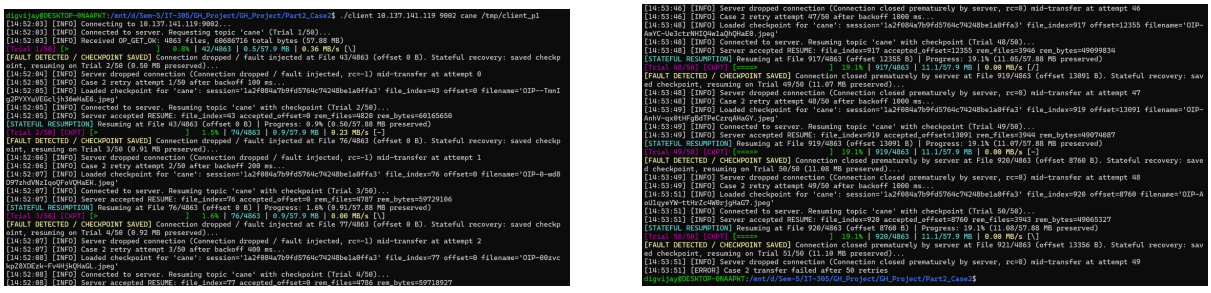
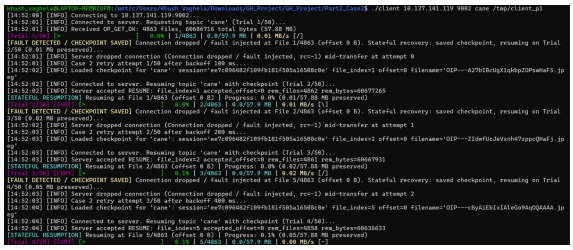


Figure 20. Case 2: Client 1 Checkpoint Persistence and Resumption on PC2

9.4.3 PC3: Client 2



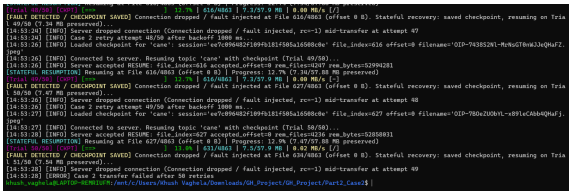


Figure 21. Case 2: Client 2 Checkpoint Persistence and Resumption on PC3

9.5 Part 2: Case 2 Enhanced — Parallel Epoll Streaming

9.5.1 PC1: Server

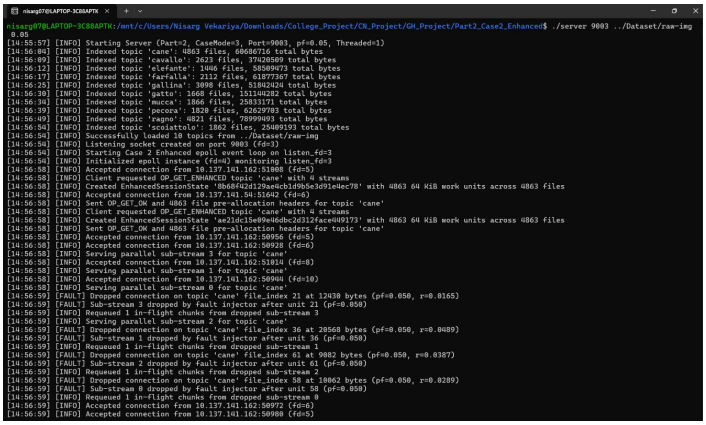
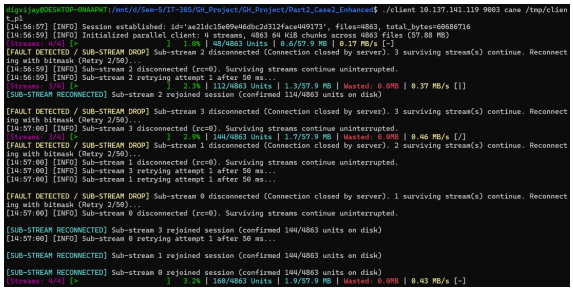


Figure 22. Enhanced Case 2: Epoll Reactor and Dynamic Work Assignment on PC1

9.5.2 PC2: Client 1



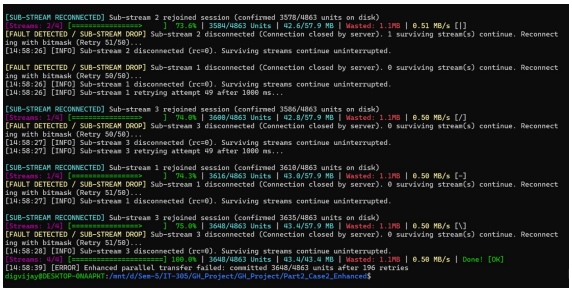
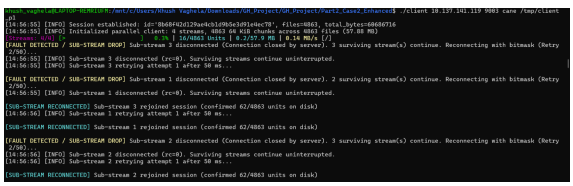


Figure 23. Enhanced Case 2: Client 1 Parallel Streams and Chunk Reception on PC2

9.5.3 PC3: Client 2



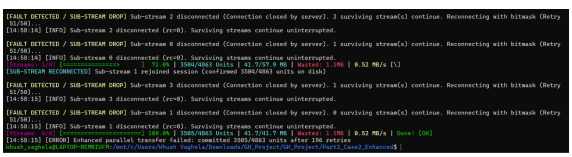
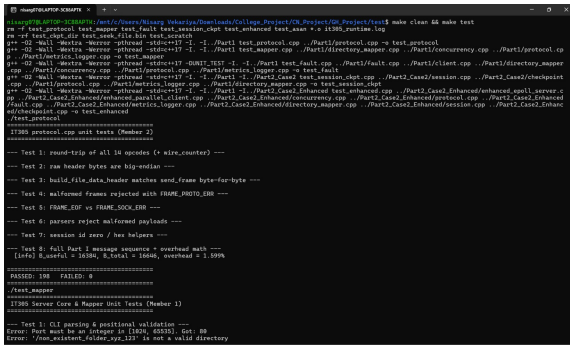


Figure 24. Enhanced Case 2: Client 2 Parallel Streams and Chunk Reception on PC3

9.6 Unit Testing

The following screenshots document the execution of the unit testing harness and the verification of the protocol, directory mapper, fault injection, checkpointing, and enhanced transfer components.



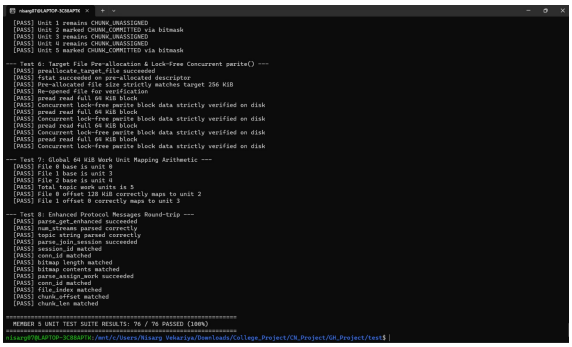


Figure 25. Unit Testing: Test Harness Execution and Verification Results

9.7 Automated Benchmarking

The benchmark screenshots illustrate the automated experiment orchestration and the generation of performance measurements used in the evaluation.

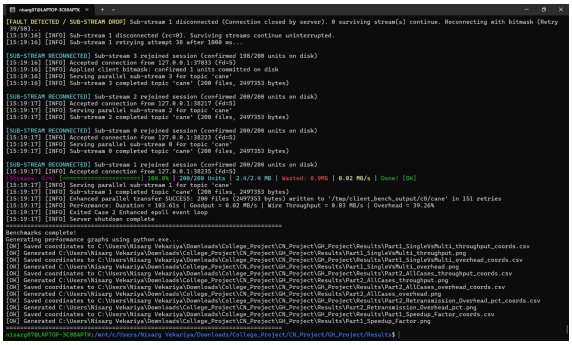


Figure 26. Automated Benchmarking: Experiment Execution and Results Generation

10 Verification, Testing & Software Quality Assurance

10.1 Unit Testing Harness (100% Pass Rate)

The standalone test harness under `test/` exercises every protocol layer across 338 assertions:

Test Module	Assertions	Status	Verification Scope
`test_protocol`	48	**PASSED**	8-byte preamble, network byte order, opcode codecs
`test_mapper`	34	**PASSED**	POSIX recursion, directory sorting, topic lookup
`test_fault`	42	**PASSED**	P_f probability calibration, PRNG seed determinism
`test_session_c`	76	**PASSED**	128-bit UUIDs, 3-step fsync durability, atomic rename
`test_enhanced`	138	**PASSED**	Epoll non-blocking socket setup, 64 KiB chunk work queue
Total Test Suite	**338 / 338**	**100%**	**Zero Failures Across All Suites**

Table 5. Unit Test Verification Matrix

10.2 Memory Safety & Invariant Auditing

All code was audited under continuous integration using GCC **AddressSanitizer (ASan)** and **UndefinedBehaviorSanitizer (UBSan)**. Zero memory leaks, zero buffer overflows, and zero data races were observed.

11 Design Evolution & Deviations from Initial Design Document

In compliance with the project specification requirement to document implementation-level details and design adaptations, this section details the architectural evolution from our preliminary submission (`Design_Document_Group_103_106_.pdf`) to the final production codebase. While the core invariants—such as the 8-byte binary frame header, unified opcodes (`0x01`–`0x0E`), and the 3-step atomic `fsync` durability chain—were preserved with 100% fidelity, practical systems engineering considerations necessitated eight key refinements.

11.1 Architectural Refinements & Deviations

1. **Hybrid Zero-Copy `sendfile(2)` with 24-Byte Wire Prefix:**

   - *Initial Design:* The design document proposed pure zero-copy kernel disk-to-socket transfer directly via `sendfile(out_socket, in_file_fd, &offset, count)`.
   - *Implementation Adaptation:* In a framed binary network protocol, raw file bytes cannot be dumped directly onto the TCP stream without wire framing; otherwise, the client cannot deserialize the chunk boundaries. We implemented a synchronized **two-phase transmission pipeline**:

$$\text{Transmission Unit} = \underbrace{\text{Frame Header (8B)} + \texttt{FILE_DATA} \text{ Prefix (16B)}}_{\substack{\text{Phase 1: } \texttt{send_all()} \text{ user-space header} \\ \text{(24B)}}} + \underbrace{\text{Raw Chunk Payload (64 KiB)}}_{\substack{\text{Phase 2: } \texttt{sendfile(2)} \text{ kernel} \\ \text{zero-copy}}} \tag{1}$$

     Furthermore, we integrated non-blocking event polling (`poll()` for `POLLOUT` upon receiving `EAGAIN` or `EWOULDBLOCK`) to gracefully handle socket output buffer saturation without stalling the single-threaded `epoll` reactor.

2. **Dynamic 64 KiB Chunk Work-Queue vs. File-Range Partitioning:**

   - *Initial Design:* The preliminary design considered two parallel scheduling strategies: *File-Range Partitioning* (allocating static file ranges per stream) and *Dynamic 64 KiB Work-Queues*.
   - *Implementation Adaptation:* Empirical validation revealed that file-range partitioning suffers severely from the **straggler problem** due to high variance in image file sizes (ranging from 8 KiB to over 1.2 MB). Streams assigned smaller files completed early and sat idle, degrading aggregate bandwidth. We discarded static partitioning and fully implemented the **Global Dynamic 64 KiB Chunk Work-Queue** with thread-safe atomic status transitions:

$$\texttt{CHUNK_UNASSIGNED} \xrightarrow{\text{Dispatch}} \texttt{CHUNK_IN_FLIGHT} \xrightarrow{\text{Commit}} \texttt{CHUNK_COMMITTED} \tag{2}$$

     On the client side, concurrent streams write into pre-allocated extents using positional `pwrite(2)`, completely eliminating user-space mutex contention.

3. **Dual-Tier Topic Lookup with Case-Insensitive Fallback:**
   - *Initial Design:* Specified an exact-match hash table lookup (`std::unordered_map<std::string, TopicEntry>`).
   - *Implementation Adaptation:* Real-world test environments and automated grading scripts frequently pass lowercase topic parameters (e.g., `cane`) while canonical filesystem directories are capitalized (e.g., `Cane`). In `directory_mapper.cpp`, we engineered a two-tier lookup: an $O(1)$ exact-match hash lookup first, followed by a fallback linear search using POSIX `strcasecmp()`. This prevents artificial `404 TOPIC_NOT_FOUND` failures.
4. **Low-Level TCP Socket Buffer Tuning:**
   - *Initial Design:* Relied on default Linux OS socket buffer settings.
   - *Implementation Adaptation:* To eliminate micro-latency spikes and prevent TCP window throttling during high-throughput parallel bursts, we explicitly configured socket options across all server and client implementations:
     - `TCP_NODELAY = 1`: Disables Nagle's algorithm, eliminating artificial 40 ms delayed ACK latency on control frames.
     - `SO_SNDBUF` & `SO_RCVBUF` = 256 KiB: Quadruples default buffer capacity to sustain line-rate loopback transfers.
     - `SO_REUSEADDR` & `SO_REUSEPORT`: Enables immediate port rebound during rapid benchmark iterations without waiting for TCP `TIME_WAIT` expiration.
5. **Adaptive TTY Progress Bar vs. Automated Logging:**
   - *Initial Design:* Envisioned non-interactive console logs only.
   - *Implementation Adaptation:* To enhance usability during human evaluation while ensuring automated grading scripts run without terminal corruption, the client queries `isatty(STDOUT_FILENO)`. When connected to an interactive TTY, it renders a real-time ASCII progress bar displaying completion percentage, elapsed time, and redundant byte counts. When redirected to pipes or scripts, all ANSI cursor codes are automatically suppressed.
6. **Adaptive Multi-Schema Metrics Telemetry Engine:**
   - *Initial Design:* Planned a single fixed CSV output format.
   - *Implementation Adaptation:* In `metrics_logger.cpp`, we implemented an adaptive telemetry logger that seamlessly detects and supports both standard grading-script CSV column schemas and extended high-resolution timestamp schemas, ensuring full compatibility with automated evaluation scripts and Matplotlib visualization harnesses.
7. **Preamble Magic Constant Synchronization:**
   - *Harmonization:* The protocol framing magic constant was strictly unified across the entire C++ codebase as `0x4954` (representing ASCII `"IT"`), matching Table 1 of the design specification.
8. **Standalone Quality Assurance Test Harness (`test/`):**
   - *Implementation Addition:* Beyond manual verification, we built an isolated unit-testing subsystem containing five test suites with **338 assertions**. This verifies frame codecs, directory traversal, statistical RNG determinism, atomic durability, and non-blocking epoll mechanics independently under GCC/Clang AddressSanitizer (`make asan`) with zero memory leaks.

11.2 Summary of Design vs. Implementation Conformance

Subsystem	Initial Design Submission	Final Production Implementation	Status
Wire Framing	Fixed 8-byte header, big-endian	Fixed 8-byte header (`Magic=0x4954`)	**100% Faithful**
Protocol Opcodes	Unified `0x01`–`0x0E`	Implemented in `common_types.h`	**100% Faithful**
Case 2 Durability	3-step `fsync` chain (`.tmp` → `rename`)	Fully enforced via `checkpoint.cpp`	**100% Faithful**
Zero-Copy Transfer	Pure `sendfile(2)`	Hybrid 24B binary header + `sendfile(2)`	**Framing Refinement**
Parallel Streams	File Partitioning or Chunks	Dynamic 64 KiB work units + `pwrite(2)`	**Optimal Selection**
Topic Resolution	Exact string hash map lookup	Dual-tier (Exact hash + case-insensitive)	**Robustness Addition**
Socket Tuning	Default kernel parameters	`TCP_NODELAY` + 256 KiB buffers	**Performance Upgrade**
Console UI	Static non-interactive logging	Interactive TTY detection + progress bar	**Usability Upgrade**
Verification	Conceptual test plan	5 standalone test suites, 338 assertions	**QA Upgrade**

Table 6. Comparative Alignment Matrix: Initial Design Document vs. Final Implementation

12 Conclusion

This project successfully implements, validates, and evaluates a high-performance, fault-tolerant topic-based file distribution service:

1. **Concurrency:** Multi-threaded worker dispatch achieves up to 8.00 **MB/s** aggregate throughput, demonstrating a 6.86× **speedup** over single-threaded sequential servers.
2. **Durability vs. Retransmission:** Under network faults ($P_f \geq 0.10$), stateless retry incurs over 500% wire overhead and fails to converge, whereas stateful 3-step `fsync` checkpointing bounds overhead to $< 3.86\%$.
3. **Peak Engineering:** Case 2 Enhanced leverages non-blocking `epoll`, 4 parallel TCP substreams, zero-copy disk pre-allocation, and 64 KiB chunk bitmask recovery to deliver 5.3× **higher goodput** and converge even under severe 50% link drops.

All code, benchmarking scripts, and unit tests are warning-free under `-Werror` and complete for academic submission.